

Actividad Practica Integradora 2

Materia: Logica de programacion

Fecha de entrega: 18/04/25

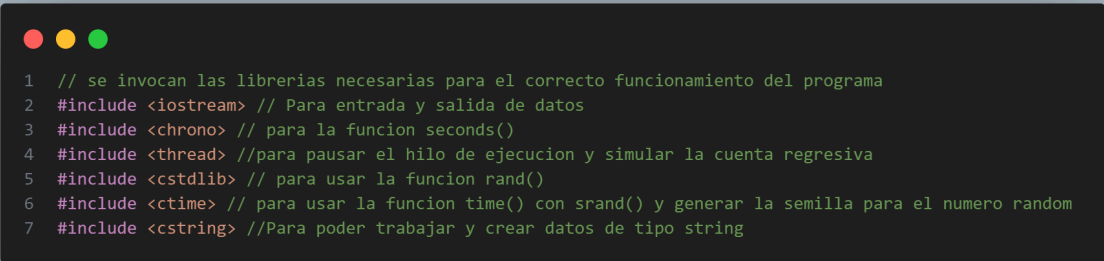
Alumno: Renzo agustin vallejo - grupo 1

Introducción:

En esta explicación que quiero abordar de la forma más clara posible, quiero aclarar que al código le he hecho modificaciones en lo que respecta a la formula que genera el número aleatorio para una mayor eficacia. Tambien se re-estructuró el código para mejorar la legibilidad del mismo, optimizando los comentarios haciéndolos más detallados sobre lo que hace cada parte del codigo y asegurandome que cada definición de variable y función sea lo mas clara posible.

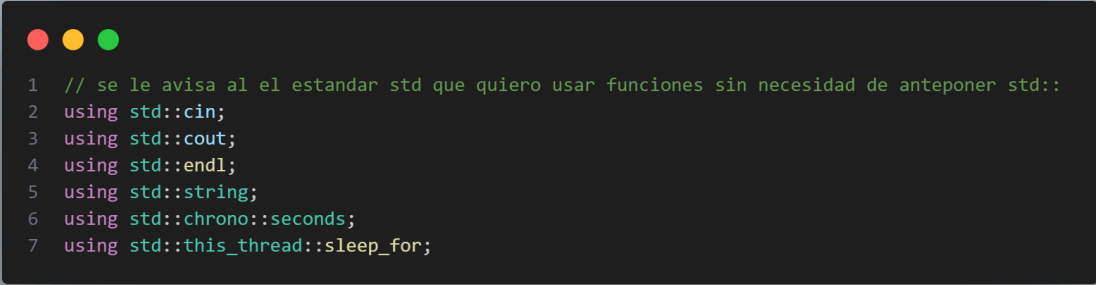
Explicacion del codigo:

1. Se cargan las librerías a utilizar en el codigo



```
1 // se invocan las librerias necesarias para el correcto funcionamiento del programa
2 #include <iostream> // Para entrada y salida de datos
3 #include <chrono> // para la funcion seconds()
4 #include <thread> //para pausar el hilo de ejecucion y simular la cuenta regresiva
5 #include <cstdlib> // para usar la funcion rand()
6 #include <ctime> // para usar la funcion time() con srand() y generar la semilla para el numero random
7 #include <cstring> //Para poder trabajar y crear datos de tipo string
```

2. Se configura la directiva using para poder usar funciones std::



```
1 // se le avisa al el estandar std que quiero usar funciones sin necesidad de anteponer std::
2 using std::cin;
3 using std::cout;
4 using std::endl;
5 using std::string;
6 using std::chrono::seconds;
7 using std::this_thread::sleep_for;
```

3. Creación de funciones propias que uso mas adelante en el codigo, a continuacion explico cada una de ellas:

```
1 void printMsgFin()
2 {
3     /*
4     -cin.ignore() Limpia el buffer para ignorar el salto de linea que es interpretado como un "enter"
5     -cin.get() Evita que se cierre la consola
6     -Se reemplazo system("pause") de windows por cin.get() para hacer el codigo mas portable ya que el comando system("pause") es propio de windows
7     */
8     cout << "-----FIN DEL JUEGO-----" << endl;
9     cout << "presiona enter para cerrar el juego...";
10    cin.ignore(1000, '\n');
11    cin.get();
12 }
13 // Funcion que valida que los datos ingresados sean los que se esperan
14 // La funcion retorna un string "ok" si todo esta bien, de lo contrario si cin.fail() es true, retorna un mensaje de advertencia
15 string validarDatos()
16 {
17     if (cin.fail())
18     {
19         cin.clear(); // Se borra el error en cin para que esta funcion se pueda volver a usar normalmente
20         cin.ignore(1000, '\n'); // Se "Vacía" el cin ignorando cualquier caracter introducido anteriormente o cualquier salto de linea
21         cout << "Haz introducido un tipo de dato invalido. Tenes que ingresar numeros." << endl;
22         return "no";
23     }
24     else
25     {
26         return "yes";
27     }
28 }
29 }
30 }
31 }
```

- 3.1. Esta función fue creada para mostrar el mensaje final en pantalla como bien lo anuncia su nombre, este procedimiento que no retorna nada, nos ayuda a reutilizar y ahorrarnos lineas de código repetidas a lo largo del proyecto:

```
1 void printMsgFin()
2 {
3     /*
4     -cin.ignore() Limpia el buffer para ignorar el salto de linea que es interpretado como un "enter"
5     -cin.get() Evita que se cierre la consola
6     -Se reemplazo system("pause") de windows por cin.get() para hacer el codigo mas portable ya que el comando system("pause") es propio de windows
7     */
8     cout << "-----FIN DEL JUEGO-----" << endl;
9     cout << "presiona enter para cerrar el juego...";
10    cin.ignore(1000, '\n');
11    cin.get();
12 }
```

- 3.2. La siguiente función que vemos en pantalla, vemos que es una función que retorna un valor string, como bien dice su nombre, está hecha para validar datos almacenados en el buffer del cin, cuenta con un if que evalua si cin.fail() es true y a partir de ello muestra un mensaje de advertencia y retorna "no", de lo contrario retorna "yes", por que retornan string? podrian retornar un bool, pero quería probar usar la libreria string, más adelante uso este valor para un condicional:

```
1 string validarDatos()
2 {
3     if (cin.fail())
4     {
5         cin.clear(); // Se borra el error en cin para que esta funcion se pueda volver a usar normalmente
6         cin.ignore(1000, '\n'); // Se "Vacía" el cin ignorando cualquier caracter introducido anteriormente o cualquier salto de linea
7         cout << "Haz introducido un tipo de dato invalido. Tenes que ingresar numeros." << endl;
8         return "no";
9     }
10    else
11    {
12        return "yes";
13    }
14 }
```

4. Esta es la funcion main y asi es como inicia, con un saludo al jugador y advirtiendolo que necesitaremos que introduzca datos antes de empezar el juego:

```
1 // creacion de la funcion principal donde se ejecuta el codigo principal del programa
2 int main()
3 {
4     /*
5     -Primero se crean las variables necesarias
6     -luego se inicializan con un valor de 0 para evitar valores basura que hayan quedado en la memoria
7     -Decidi crearlas todas en una misma linea porque se me hizo mas practico por las lineas de codigo ahorradas.
8     */
9     int cantidad_intentos = 0, limite_inferior = 0, limite_superior = 0, numero_aleatorio = 0, posible_numero = 0;
10    int intentos_realizados = 0, intentos_restantes = 0;
11
12    /*
13    A continuacion se da una breve explicacion de que trata el juego y se procede
14    a solicitar los datos necesarios al jugador, cada dato es almacenado en su respectiva variable
15    */
16
17    cout << "-----INICIA EL JUEGO-----" << endl;
18    cout << "Bienvenido al juego de adivinanzas, donde el objetivo es que adivines el numero generado de forma aleatoria antes de que se te acaben los intentos :)" << endl;
19
20    cout << "\nAntes de empezar tenes que indicar dentro de que rango de numeros quieres que generemos un numero aleatorio para vos" << endl;
21
```

5. A continuacion tenemos 3 bucles while, que ejecutan codigo parecido, los 3 bucles invocan a la función validarDatos(), la ya explicada anterior mente que retorna un string, aqui reside la razón por la cual retorna un string, su retorno es usado en el condicional if dentro del while:

```
1 /*
2 -El bucle while se ejecutara mientras no se rompa su flujo de ejecucion ya que la condicion esta establecida como true,
3 lo que ocasionara que se repita la pregunta hasta que el usuario ingrese un tipo de dato valido y se ejecute el break;
4 */
5 while (true)
6 {
7
8     cout << "ingresa el limite inferior: ";
9     cin >> limite_inferior;
10
11    // if que evalua lo que retorna la funcion validarDatos()
12    if (validarDatos() == "yes")
13    {
14
15        if (limite_inferior < 0)
16        {
17            cout << "El limite inferior no puede ser menor a 0" << endl;
18        }
19        else
20        {
21            break; // rompe el while y la ejecucion vuelve al scope global
22        }
23    }
24 }
25
26 while (true)
27 {
28
29     cout << "Ahora ingresa el limite superior: ";
30     cin >> limite_superior;
31
32    // if que evalua lo que retorna la funcion validarDatos()
33    if (validarDatos() == "yes")
34    {
35
36        if (limite_superior < limite_inferior)
37        {
38            cout << "El limite superior no puede ser menor al limite inferior" << endl;
39        }
40        else if (limite_superior == limite_inferior)
41        {
42            cout << "El limite superior no puede ser igual al limite inferior, debe ser mayor" << endl;
43        }
44        else
45        {
46            break; // rompe el while y la ejecucion vuelve al scope global
47        }
48    }
49 }
50
51 while (true)
52 {
53
54     cout << "\nOk. Cuantos intentos quieres tener? pueden ser tanto como quieras: ";
55     cin >> cantidad_intentos;
56
57    // if que evalua lo que retorna la funcion validarDatos()
58    if (validarDatos() == "yes")
59    {
60
61        if (cantidad_intentos <= 0) // cantidad_intentos no puede ser igual o menor que 0
62        {
63            // si cantidad_intentos es igual o menor que 0, se muestra un mensaje al usuario y se ejecuta el bucle hasta que introduzca un cantidad valida
64            cout << "La cantidad de intentos maximos debe ser mayor a 0" << endl;
65        }
66        else
67        {
68            break; // rompe el while y devuelve el control de la ejecucion al scope global
69        }
70    }
71 }
```

6. Luego continua la ejecución con un par de líneas de salida de datos que imprimen en pantalla los siguientes mensajes mostrando los datos que el usuario ha ingresado:

```
1 cout << "\nMuy bien, empezaste excelente, elegiste tener" << " " << cantidad_intentos << " " << "intentos" << endl;
2 cout << "y el rango que estableciste va desde " << limite_inferior << " hasta " << limite_superior << endl
3     << endl;
4
5 cout << "Bueno, prepárate porque el juego empieza en: " << endl;
```

7. Luego se ejecuta un for que simula un conteo regresivo que da tiempo a que el jugador se prepare. Cuando termina el conteo inmediatamente se crea la semilla que servirá para que posteriormente se genere el número random con la fórmula que se puede observar en la imagen:

```
1 for (int i = 5; i >= 1; i--)
2 {
3     sleep_for(seconds(1));
4     cout << i << endl;
5 }
6
7 // Con srand() se inicializa la función rand(), generando una semilla basada en el reloj para que rand() no repita la misma secuencia de números aleatorios
8 srand(time(0));
9
10 /*
11  -se corrigió la fórmula para mayor eficacia
12  -Además ahora la fórmula se colocó fuera del bucle
13  para generar solo un número random y no varios,
14  esto resulta en una mayor probabilidad de adivinar
15  -fórmula para generar número random en un rango específico
16  */
17 numero_aleatorio = rand() % (limite_superior - limite_inferior + 1) + limite_inferior;
18
19 /*
20  Este bucle for es el que ejecuta la secuencia de bloques por cada intento del jugador
21  contiene un condicional if en su interior que se ejecuta solo si el usuario adivina el número
22  */
```

8. Luego se ejecuta el bucle for que es el más importante ya que contiene la mayor parte del código, bucles, condicionales para evaluar los datos con sus respectivas proposiciones y los

contadores que serán útiles para limitar los intentos del jugador y poder darle un fin al programa:

```
1  for (int z = cantidad_intentos; z > 0; z--) // Condicion que hace que el bloque se ejecute determinadas veces
2  {
3      // Por cada vuelta del bucle for se va a evaluar si el jugador esta introduciendo un tipo de dato valido
4      do
5      {
6          cout << "Cual es el numero secreto?: ";
7          cin >> posible_numero;
8
9          /*
10         -Cual el jugador introduzca un dato se evaluara si hay un error con el tipo de dato ingresado haciendo uso de cin.fail()
11         -Si cin.fail() detecta un tipo de dato invalido su valor sera true entonces se ejecutara el if mostrando un mensaje de advertencia,
12         luego se borrara el valor de cin con cin.clear()
13         por ultimo con cin.ignore() se eliminan los caracteres introducidos
14         o cualquier salto de linea que haya quedado guardado en el para evitar su ejecucion y que nos siga mostrando fallos
15         */
16         if (cin.fail())
17         {
18             cout << "Debes introducir numeros. otros caracteres no son validos." << endl;
19             cin.clear();
20             cin.ignore(1000, '\n');
21
22             // Cada dato mal introducido tambien se contara como intento fallido y se le mostrara la informacion en pantalla
23             intentos_realizados++;
24             z--; /*Es probable que el while se ejecute varias veces y el for quede pausado,
25             esto ocasionara que z no se decremente tras cada vuelta del ciclo while, lo que podria generar un problema
26             si el jugador de repente introduce un dato valido ya que tendria mas intentos de los que
27             les corresponden, entonces es necesario ir decrementando z manualmente dentro del while para que cuando se
28             rompa el bucle y el control de la ejecucion pase al scope del for
29             el jugador solo cuente con los intentos restantes teniendo en cuenta los que ya gasto dentro del scope del while*/
30
31             cout << "Intentos realizados: " << intentos_realizados << endl;
32             cout << "Intentos restantes: " << cantidad_intentos - intentos_realizados << endl;
33             cout << "-----" << endl;
34
35             if (z == 0)
36             {
37                 cout << "Intentos agotados. solo tenias " << cantidad_intentos << ". Suerte para la proxima" << endl;
38                 printMsgFin();
39                 return 0; // Finaliza la ejecucion del codigo
40             }
41         }
42         else
43         {
44             // Si el jugador ingreso un tipo de dato valido entonces se rompe el flujo del while para continuar con el flujo del for
45             intentos_realizados++;
46             break;
47         }
48     } while (true);
49
50
51     /*
52     En caso de que el jugador adivine el numero , el condicional ejecuta el mensaje cout
53     y lanza un return 0 para finalizar la ejecucion del codigo instantaneamente.
54     */
55     if (posible_numero == numero_aleatorio)
56     {
57         cout << "INCREIBLEEEEE!!!! LOGRASTE ADIVINAR EL NUMERO SECRETO :D" << endl;
58         cout << "Tenias " << cantidad_intentos << " intentos ";
59         cout << "y has realizado " << intentos_realizados << endl;
60
61         printMsgFin();
62         return 0; // Finaliza la ejecucion del codigo porque termino el juego
63     }
64
65     /*
66     -Si el jugador introduce un numero y no adivino el numero secreto entonces se evalua si el numero introducido cumple con las condiciones establecidas
67     */
68     if (posible_numero > limite_inferior && posible_numero < limite_superior)
69     {
70
71         // Si se encuentra dentro del rango establecido entonces se procede a evaluar si es mayor al numero secreto
72         if (posible_numero > numero_aleatorio)
73         {
74             cout << "El numero que ingresaste es mayor que el numero secreto" << endl;
75         }
76         else if (posible_numero < numero_aleatorio) // Si se encuentra dentro del rango establecido y no es mayor al numero secreto entonces se evalua si es menor
77         {
78             cout << "El numero que ingresaste es menor que el numero secreto" << endl;
79         }
80     }
81
82     /*
83     En caso de que el numero no se encuentre dentro del rango establecido,
84     se evalua si el numero ingresado esta por debajo del limite inferior o por encima del limite superior
85     */
86     else if (posible_numero < limite_inferior)
87     {
88         cout << "El numero que ingresaste no se encuentra dentro del rango establecido por que es inferior a " << limite_inferior << endl;
89     }
90     else if (posible_numero > limite_superior)
91     {
92         cout << "El numero que ingresaste no se encuentra dentro del rango establecido por que es mayor a " << limite_superior << endl;
93     }
94
95     cout << "Intentos realizados: " << intentos_realizados << endl;
96     cout << "Intentos restantes: " << cantidad_intentos - intentos_realizados << endl;
97     cout << "-----" << endl;
98 }
```

Comentario sobre el código: Si bien el código cumple con lo pedido para la actividad 2, este código puede mejorarse y optimizarse, reducir líneas de código, diferentes formas de estructurarlo y darle más estética al programa en la consola, lo cual tengo pensado hacer para la actividad del módulo 3. Cabe destacar que me apoyé tanto del material de estudio brindado por los tutores como de la documentación oficial de C++ proporcionada por Microsoft y también de otras fuentes online.